PROGRAMMATION WEB DYNAMIQUE

Benoit Gaudou - benoit.gaudou@ut-capitole.fr

ORGANISATION

- CM : 12h (8 x 1,5h)
- TP : 24h (8 x 3h)

- Travaux pratiques : en salle TP
  (possibilité de le faire sur les machines personnelles)

PRÉ-REQUIS

- **Web** :
  - Développement de sites web statiques (HTML5)
  - Mise en forme / Feuilles de styles (CSS 3)
  - Comportement (Javascript)
- **Programmation**
  - Java
- **Base de données**
  - SQL
- **Note : 1 CM et 1 TP de révision de la partie Web**

EVALUATION

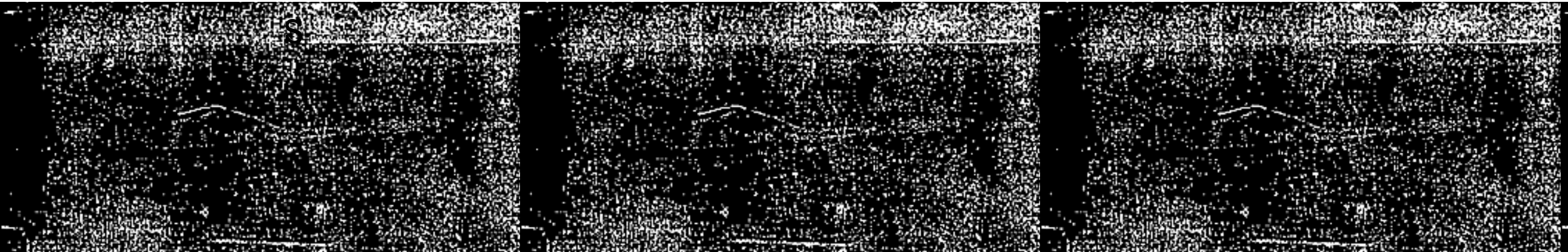
- **Examen sur table** (50%* note)
  - dernier CM (**date à revoir**)
- **QCMs au début de 1+ séance(s) de TP sur le cours** (10 %* note)
  - TP5 (19/03*) + …
- **TP noté à faire en 3h en salle machine** (20%* note)
  - Dernier TP (TP8 : 31/03*)
- **TPs à finir et à rendre** (malus TP non rendus)
  - TP1, TP2+3, TP4+5, TP6+7
- **Semaine projet web** (20%* note)

*Pourcentages et dates donnés à titre indicatif

SUPPORTS, BIBLIO, RÉFÉRENCES ET PLUS

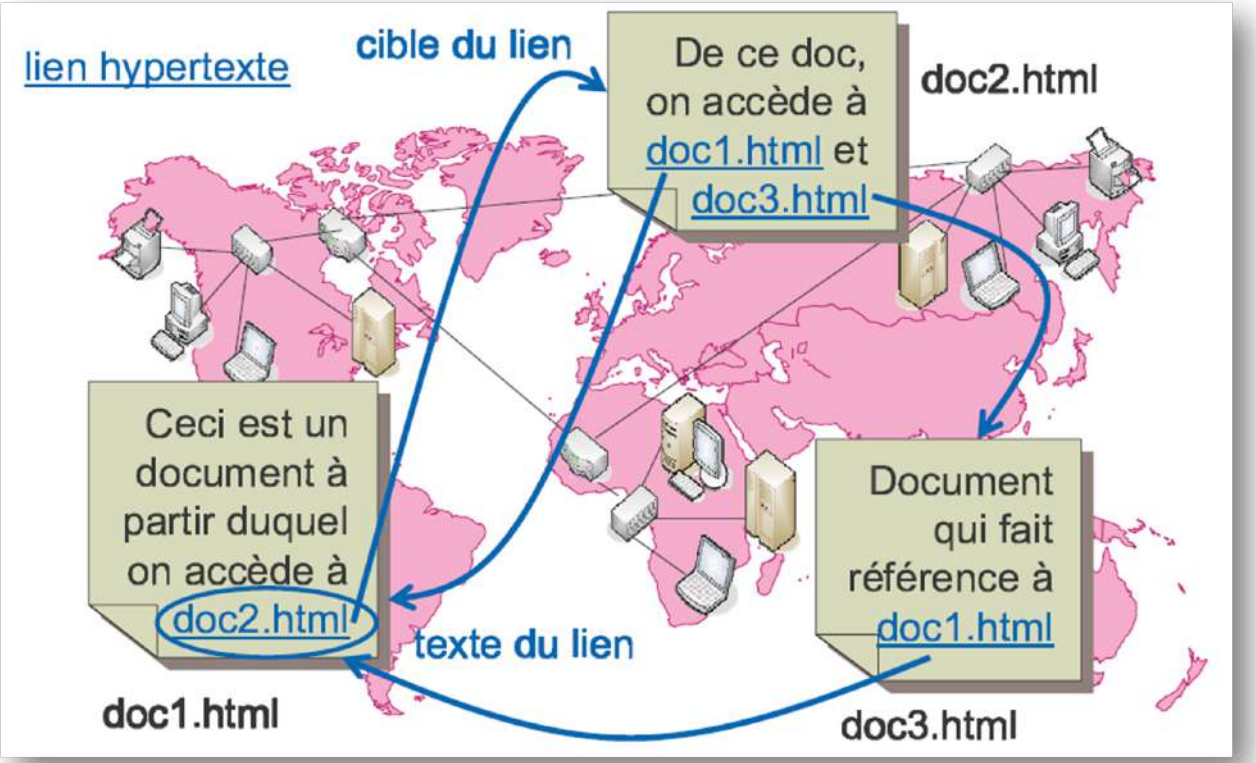
- Sur Moodle
- Web statique :
  Learning Web Design (4th edition), J. Niederst Robbins
- SpringBoot :
  https://spring.io/projects/spring-boot

RAPPEL : LE WEB ????



LE WEB

- **Le Web est un service d'internet.**
- Il permet de relier les documents entre eux en utilisant des **liens hypertextes**, formant ainsi un énorme «web» d'informations connectées.
- Basé sur la technique **hypertexte** (langage HTML et protocole HTTP)



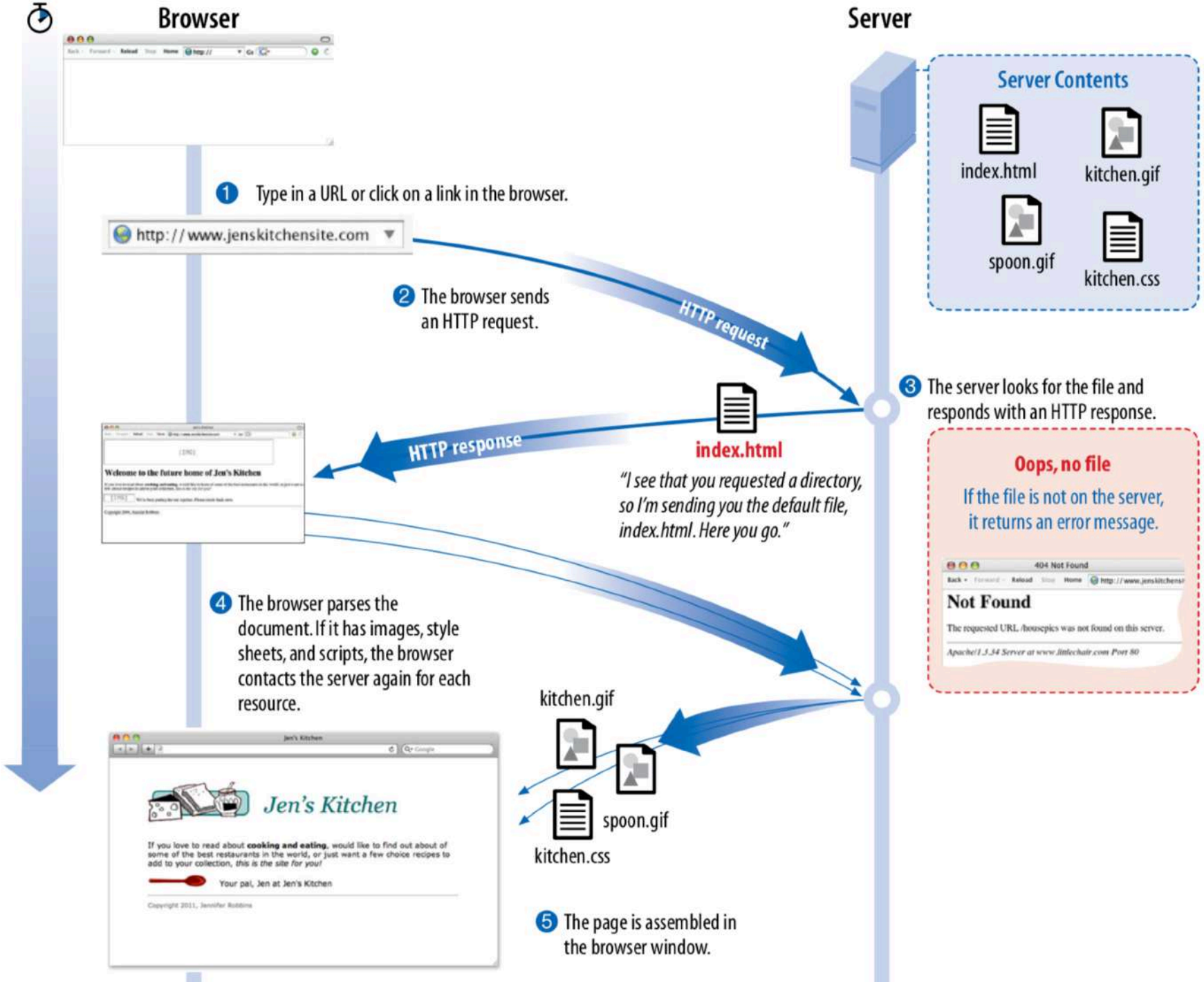
Source : Sylvie Doutre

WEB ET SERVEURS

- Parce qu'ils "servent" des documents sur demande, ces ordinateurs sont connus comme des **<u>serveurs</u>**.
- un serveur est un dispositif informatique matériel et/ou logiciel **qui offre des services, à différents clients.**
- Les services les plus courants sont :
  - l'accès aux pages du web,
  - le courrier électronique,
  - le stockage en base de données

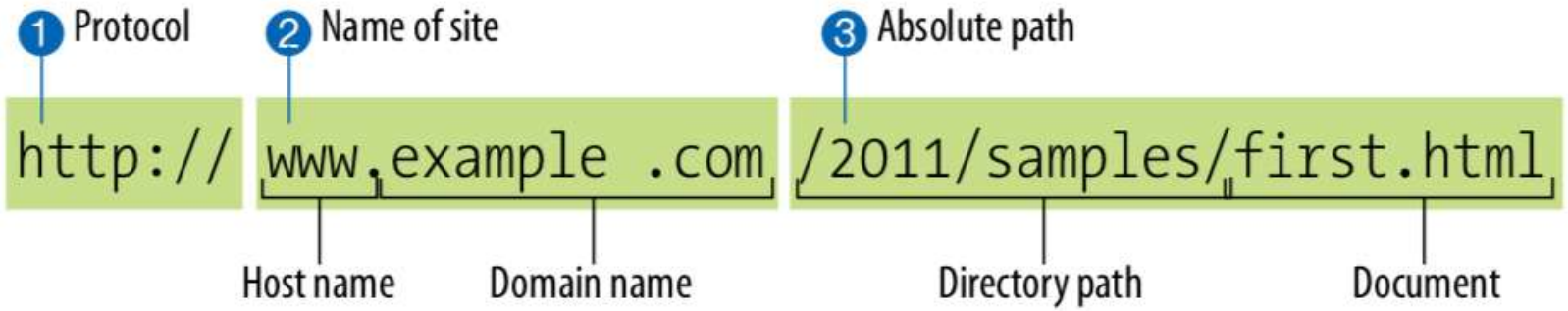
ARCHITECTURE

- Le logiciel qui fait la demande s'appelle **le client** (souvent le navigateur)
- On utilise un navigateur pour accéder aux documents sur le Web
- Le serveur renvoie les documents pour que le navigateur affiche les requêtes et les réponses sont traitées via le protocole HTTP
- HTTP peut être utilisé pour transférer des images, des films, des fichiers audio, des données, des scripts, …


Browser
Server
Server Contents
index.html
kitchen.gif
spoon.gif
kitchen.css
1 Type in a URL or click on a link in the browser.
http://www.jenskitchensite.com
2 The browser sends an HTTP request.
HTTP request
3 The server looks for the file and responds with an HTTP response.
index.html
"I see that you requested a directory, so I'm sending you the default file, index.html. Here you go."
HTTP response
Oops, no file
If the file is not on the server, it returns an error message.
404 Not Found
Not Found
The requested URL /housepics was not found on this server.
4 The browser parses the document. If it has images, style sheets, and scripts, the browser contacts the server again for each resource.
kitchen.gif
spoon.gif
kitchen.css
Jen's Kitchen
5 The page is assembled in the browser window.

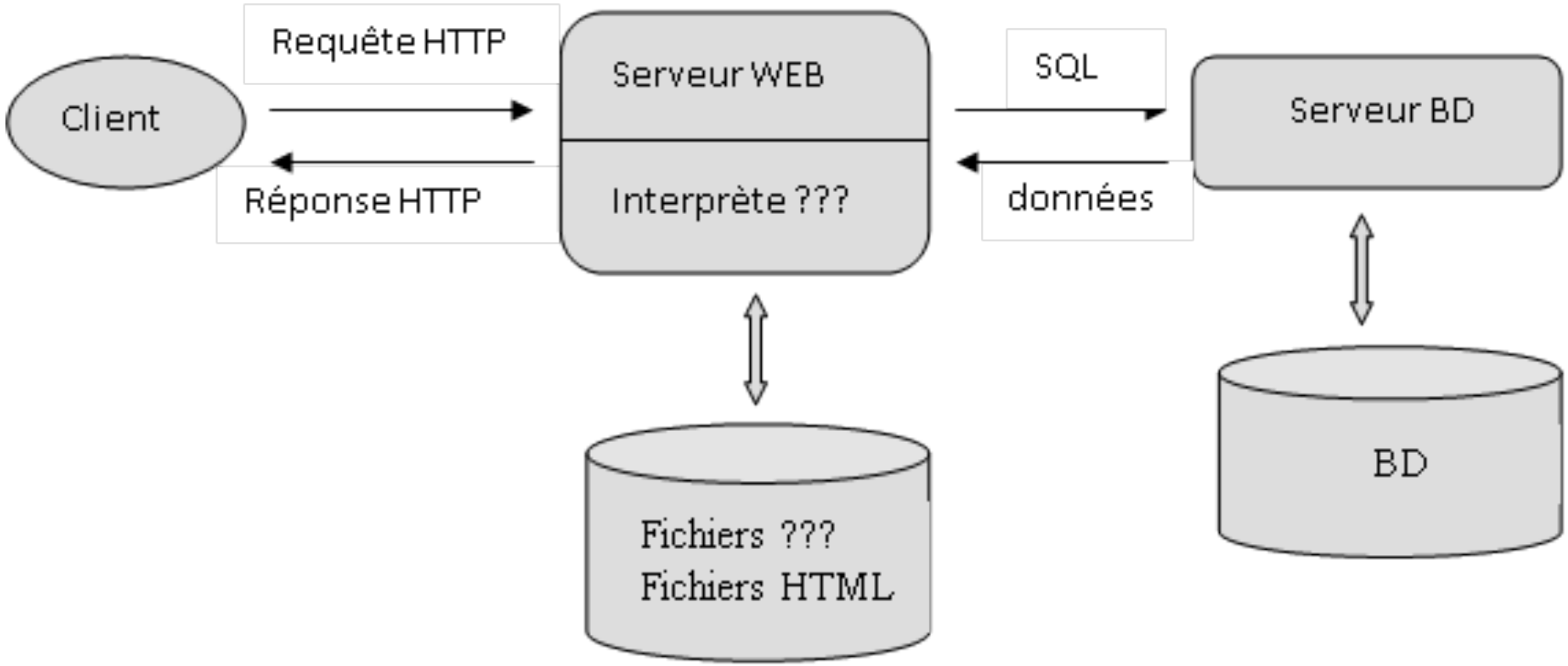
WEB PAGE ADRESSES (URLS)

- Chaque page et ressource sur le Web a sa propre adresse spéciale appelée **URL** (Uniform Resource Locator)

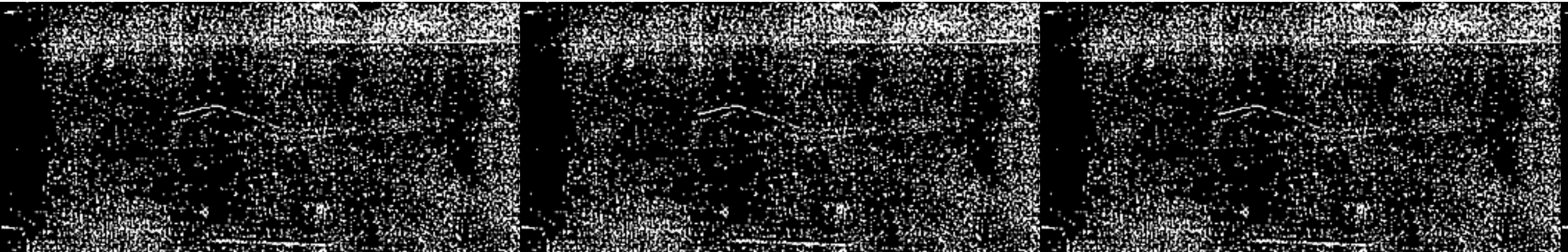


- Pour résumer, l'URL ci-dessus dit qu'il aimerait utiliser le protocole HTTP pour se connecter à un serveur Web sur Internet appelé www.example.com et demander le document first.html (situé dans le répertoire samples, qui est dans le répertoire 2011).

ARCHITECTURE AVANCÉE



QUELS LANGAGES ?



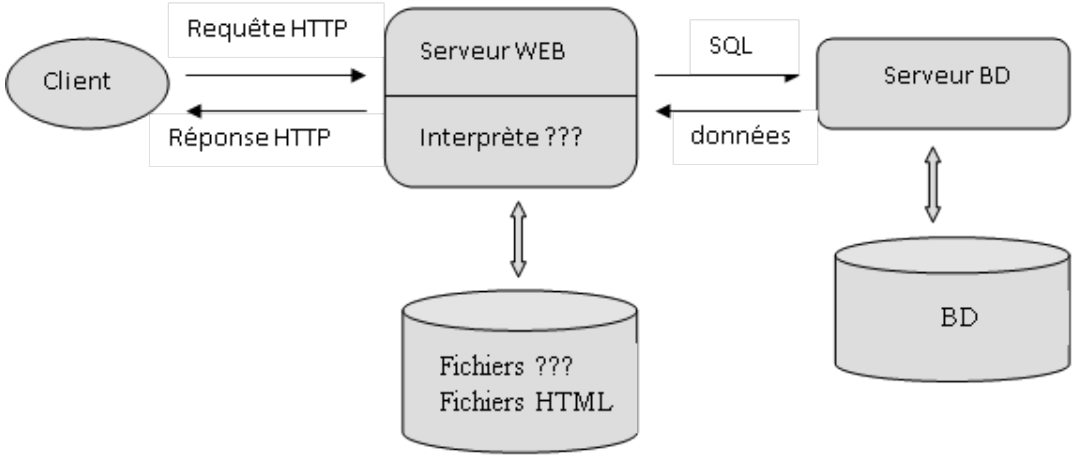
FRONTEND VS BACKEND

- **Conception Frontend**
  - Conception se rapportant directement au navigateur
  - Conception graphique et production d'images
  - Conception d'interface
  - Conception de l'information liée à l'expérience de l'utilisateur sur le site

  - Développement de document **HTML** et feuille de style **CSS**
  - Interaction en **JavaScript**

FRONTEND VS BACKEND

- "**Backend**" fait référence aux programmes et scripts qui fonctionnent sur le serveur en arrière-plan pour rendre les pages web dynamiques et interactives.
  - Organisation de l'information sur le serveur
  - Traitement de formulaires
  - Interrogation de base de données et traitement des résultats
  - Autres applications Web côté serveur utilisant **PHP**, JSP, Ruby, ASP.NET, **Java**, Python, Node.js etc…



HYPERTEXT MARKUP LANGUAGE (HTML)

- HTML 5
  - implémentation plus stricte XHTML (eXtensible HTML), essentiellement le même langage mais avec des règles de syntaxe plus strictes
- **HTML n'est pas un langage de programmation c'est un langage de balisage**
- **Décrit la structure sous-jacente du document**

Jen's Kitchen.html UNREGISTERED

Jen's Kitchen.html

```html
<!DOCTYPE html>

<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=windows-1252">
        <title>Page de Jen's Kitchen</title>
        <link rel="stylesheet" href="./Jen&#39;s Kitchen_files/kitchen.css">
    </head>

    <body data-new-gr-c-s-check-loaded="14.991.0" data-gr-ext-installed="">
        <h1>
            <img src="./Jen&#39;s Kitchen_files/foods.gif" alt="food illustration"> Jen’s Kitch
        </h1>

        <p>If you love to read about <strong>cooking and eating</strong>, would like to find out about of some of the best restaurants in the world, or just want a few choice recipes to add to your collection, <em>this is the site for you!</em></p>

        <p><img src="./Jen&#39;s Kitchen_files/spoon.gif" alt="spoon illustration"> Your pal, Jen at Jen's Kitchen</p>

        <hr>

        <small>Copyright 2011, Jennifer Robbins</small>
    </body>
</html>
```

Line 6, Column 24

Page de Jen's Kitchen

file:///Users/benoitgaudou/Desktop/L3 TI - WebDEV/WebProg-C3-

Jen’s Kitch

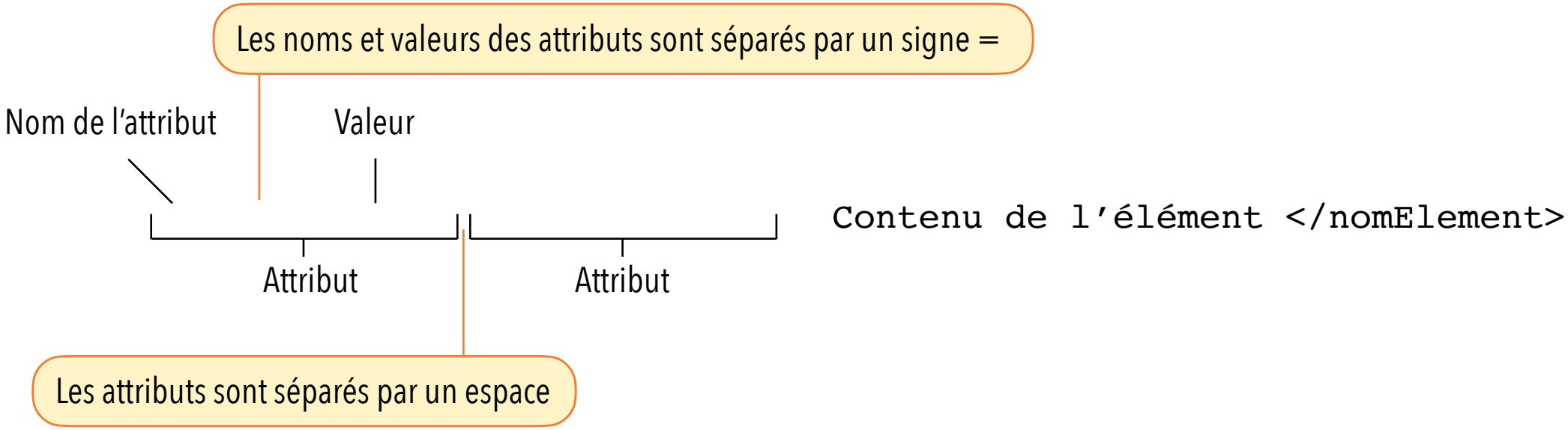
If you love to read about **cooking and eating**, would like to find out about of some of the best restaurants in the world, or just want a few choice recipes to add to your collection, *this is the site for you!*

Your pal, Jen at Jen's Kitchen

Copyright 2011, Jennifer Robbins

BALISE HTML (COMPLET)

- Les balises peuvent avoir des attributs (ex. le chemin vers l'image à afficher, l'URL d'une page pour une balise de lien …)



- Exemple :

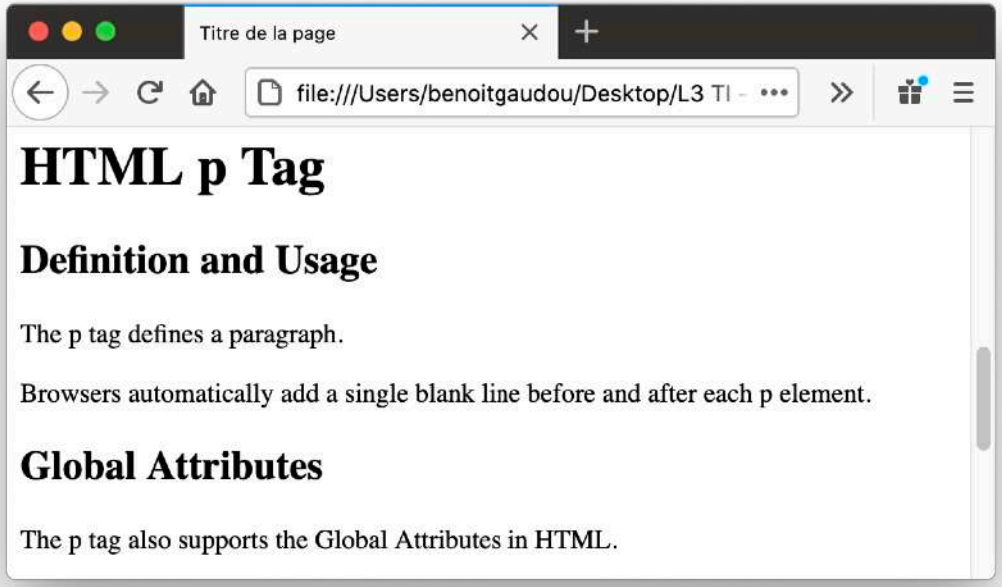
```
Université Toulouse Capitole </a>
```

STRUCTURE MINIMALE D'UNE PAGE

```
1  <!DOCTYPE html>

2  <html>

3    <head>
       <meta charset="utf-8">          4
       <title> Titre entête </title>   5
     </head>

6    <body>
       Contenu de la page ici.
     </body>

     </html>
```

TITRES ET PARAGRAPHES

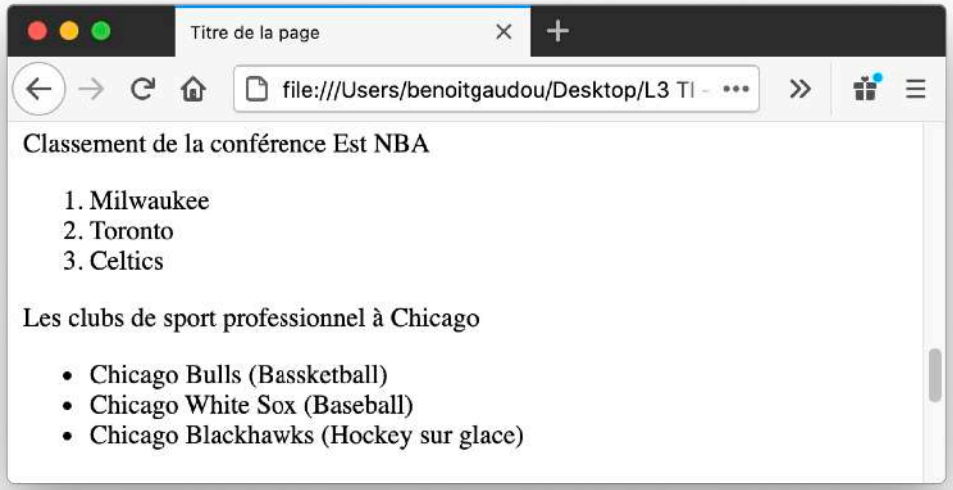
```
<h1> HTML p Tag</h1>

<h2> Definition and Usage </h2>

<p> The p tag defines a paragraph.</p>

<p>Browsers automatically add a single blank
line before and after each p element.</p>

<h2> Global Attributes </h2>

<p> The p tag also supports the Global
Attributes in HTML. <p>
```



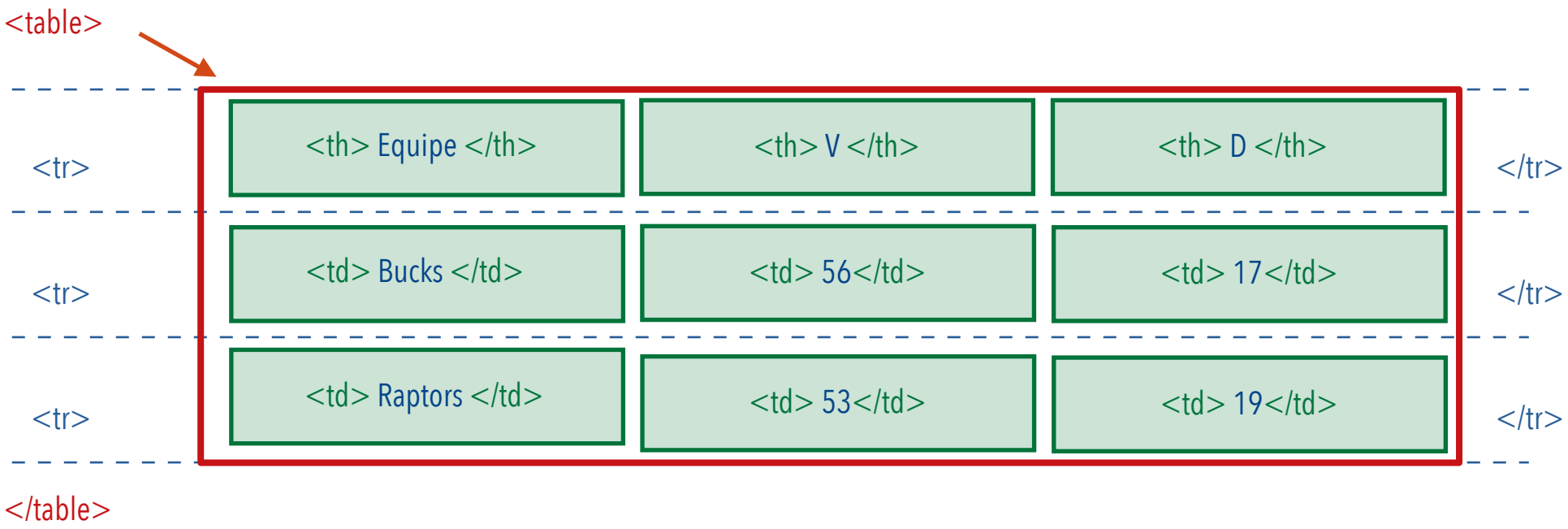
LES LISTES : UL, OL, LI

- Les listes peuvent être numérotées (ol) ou non numérotées (ul).

```
<p> Classement de la conférence Est NBA </p>

<ol>
    <li>Milwaukee</li>
    <li>Toronto</li>
    <li>Celtics</li>
</ol>

<p> Les clubs de sport professionnel à Chicago</p>
<ul>
    <li>Chicago Bulls (Basketball)</li>
    <li>Chicago White Sox (Baseball)</li>
    <li>Chicago Blackhawks (Hockey sur glace)</li>
</ul>
```

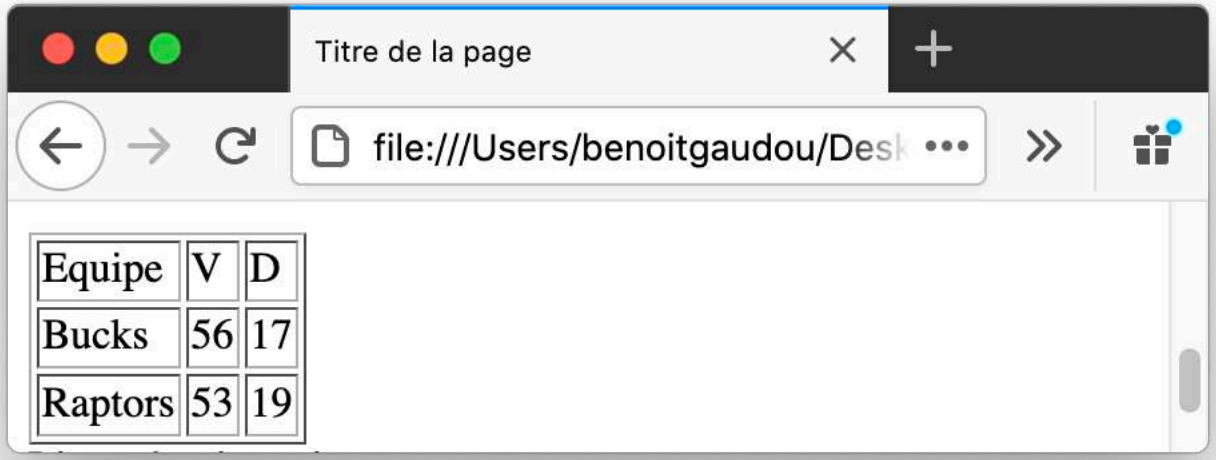


LES TABLEAUX EN HTML



LES TABLEAUX : TABLE, TR, TH, TD

```
<table border="1">
    <tr>
        <th>Equipe</th>
        <th>V</th>
        <th>D</th>
    </tr>
    <tr>
        <td>Bucks</td>
        <td>56</td>
        <td>17</td>
    </tr>
    <tr>
        <td>Raptors</td>
        <td>53</td>
        <td>19</td>
    </tr>
</table>
```



ORGANISER LE CONTENU D'UNE PAGE

- Eléments génériques **`<span> <div>`**
  - rôle totalement neutre
  - servent à regrouper plusieurs éléments HTML pour leur affecter un style CSS commun (**span : sur une portion de texte, div sur un ensemble d'éléments**)

- HTML5 : Les éléments de section
  `<section>`, `<article>`, `<nav>`, `<aside>`, `<header>`, `<footer>` segmentent des portions du document
  - Construit par analyse de la structuration principale de la majorité des pages web
  - Par exemple

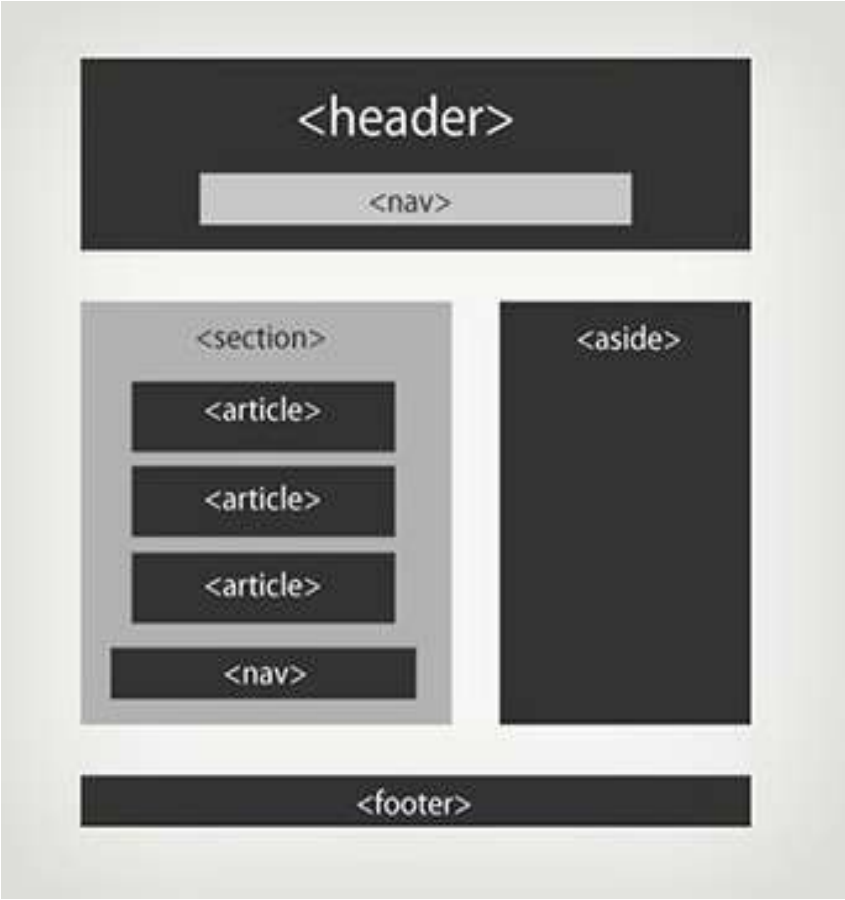
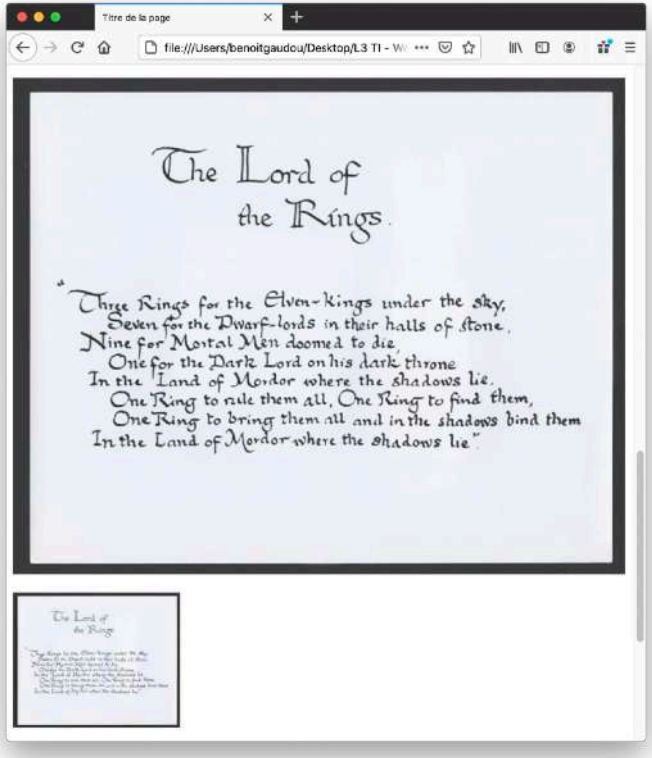


IMAGE: IMG

```
<img src="http://tolkiengateway.net/w/images/7/75/J.R.R._Tolkien_-_Ring_verse.jpg"
alt="citation du Seigneur des anneaux"/>

<img src="J.R.R._Tolkien_-_Ring_verse.jpg" alt="citation du Seigneur des anneaux"/>
```

- `src` = adresse de l'image (peut être en local ou sur internet)
- `alt` = message alternatif (en cas de manque de l'image)



LIEN HYPERTEXTE : A

- **Un lien vers une page d'un autre site Web**
  - `<a href="http://www.ut-capitole.fr"> Université Toulouse Capitole </a>`
  - Note : attribut `TARGET="_blank"` à préciser pour que la page s'ouvre dans un autre onglet/fenêtre
- **Un lien vers une page du site**
  - `<a href="page2.html"> La suite page 2 </a>`
- **Un lien vers une ancre** (= un endroit particulier dans une page, besoin d'un élément avec attribut `id="ancre"`)
  - `<a href="page2.html#ancre"> la suite page 2, paragraphe ancre </a>`
- **Un lien vers un autre service Internet (courriel, FTP)**
  - `<a href="mailto:moi@tel.com"> M'écrire </a>`

<FORM>..</FORM>

- Un formulaire se situe dans une balise **form.**
- A l'intérieur, il y a un ensemble d'**input**.

Rejoignez notre mailing list

Recevez les dernières nouvelles du retaurant

Nom :

Email :

Envoyez !

```
<form action="./mailinglist.php" method="post">
   <p>Rejoignez notre mailing list</p>
   <p>Recevez les dernières nouvelles du retaurant</p>

   Nom : <input type="text" name="nom" id="firstlast"/> <br>
   Email : <input type="text" name="mel" id="id_email" required/> <br>

  <input type="submit" value="Envoyez !">
</form>
```

ATTRIBUTS DE <FORM>

- **`action`** : indique l'adresse où seront envoyées les données du formulaire :
  - page ou script (php, asp.net…) ou route (cf. Springboot) : `" mailinglist.php", "confirmation.html", "/conference/inscr"`
  - adresse email : `"mailto:nom@domaine.ext"`

- **`method`** : comment l'information est envoyée au serveur (verbe HTTP : GET ou POST)

```
<form action="./mailinglist.php" method=« POST">
```

APPUI SUR SUBMIT ?

- Lors de l'envoi des données, la valeur entrée par l'utilisateur est associée avec le nom de l'input (attribut **name**).

Rejoignez notre mailing list

Recevez les dernières nouvelles du retaurant

1. Nom : Jean
2. Email : jean.dupont@gmail.com

Envoyez !

```
<form action="mailto:nom@domaine.ext" method="post">
   <fieldset>
         <legend>Rejoignez notre mailing list</legend>
         <p>Recevez les dernières nouvelles du retaurant</p>
         <ol>
              <li>
                   <label for="firstlast">Nom :</label>
                  <input type="text" name="nom" id="firstlast"/>
             </li>
             <li>
                   <label for="id_email">Email :</label>
                  <input type="text" name="mel" id="id_email" required/>
             </li>
         </ol>
          <input type="submit" value="Envoyez !">
   </fieldset>
</form>
```

Nouveau message

nom@domaine.ext

Objet

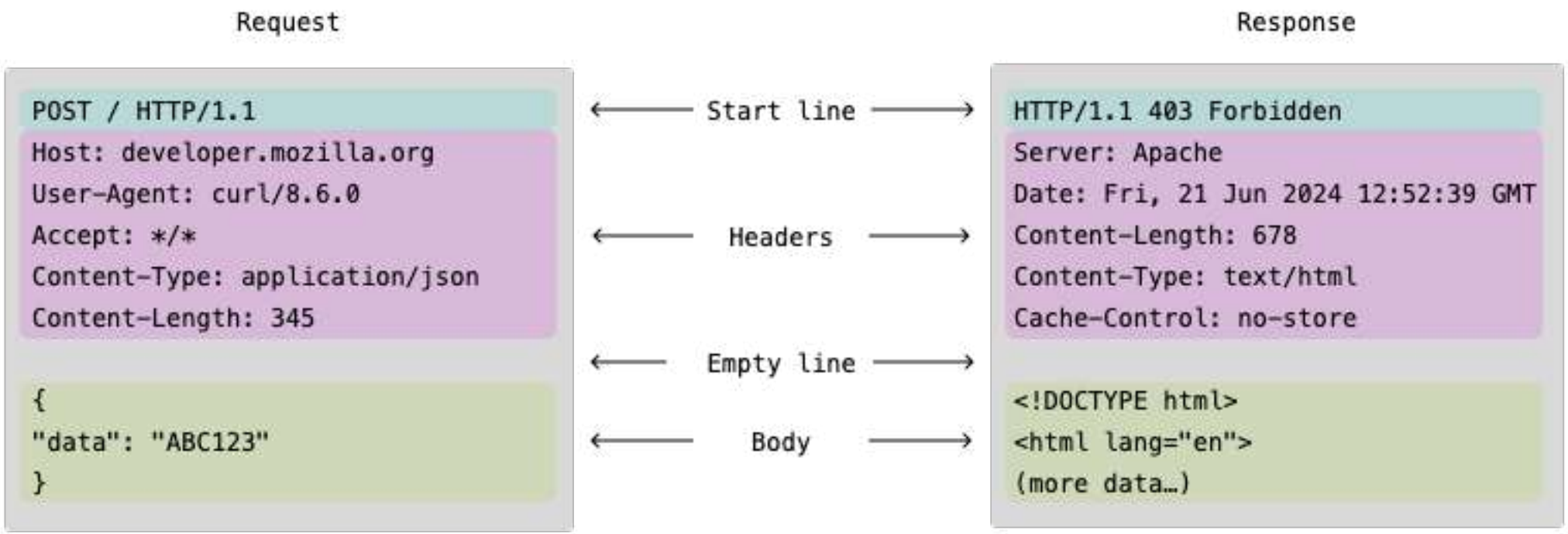
nom=Jean&mel=jean.dupont%40gmail.com

RAPPEL : PROTOCOLE HTTP

- **HTTP (HyperText Transfer Protocol)** est un protocole de communication utilisé sur le web pour transférer des données entre un client (browser…) et un serveur web (dans les 2 sens).
- HTTP est un protocol « stateless » : chaque requête est indépendante et le serveur ne garde pas en mémoire les interactions passées.

- Il se base sur le modèle **client-server**.
  - **Client**: envoie une **requête HTTP** pour demander des ressources (pages web, images, fichiers).
  - **Server**: répond à une requête HTTP en fournissant les ressources demandées (ou un message d'erreur).

EXEMPLES DE MESSAGES



https://developer.mozilla.org/fr/docs/Web/HTTP/Guides/Messages

REQUÊTE HTTP

- **Request/start Line:**
  - Contient la méthode (GET, POST, PUT…), le chemin vers la ressource, et la version d' HTTP.
  - Exemple: `GET /index.html HTTP/1.1`
- **Request Headers:**
  - Fournit des metadata sur la requête.

```
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

- **Request Body (optional):**
  - Utilisé pour envoyé des données.
  - Exemple d'une requête POST :

```
username=john&password=1234
```

RÉPONSE HTTP

- **Status/start Line:**
  - Indique le status de la requête (succès, erreur, redirection) au moyen d'un code
  - Exemple : `HTTP/1.1 200 OK`

- **Response Headers:**
  - Fournit des informations sur le serveur et les données renvoyées.
  - Exemple :

```
Content-Type: text/html
Content-Length: 123
```

- **Response Body (optionnel):**
  - Contient les ressources demandées (HTML, JSON, images, etc.).

ATTRIBUTS DE <FORM>

- **`method`** : comment l'information est envoyée au serveur
  - **GET** (par défaut): information envoyée avec l'adresse
    - `[…]/mailinglist.php?name=Sally%20Strongar m&email=strongarm%40example.com`

```
<form action="./mailinglist.php" method="get">
```

ATTRIBUTS DE <FORM>

- **`method`** : comment l'information est envoyée au serveur
  - **POST**: information envoyée encapsulée (information sécurisée ou grand volume)

```
<form action="./mailinglist.php" method="post">
```

```
POST /mailinglist.php HTTP/1.1
Host: foo.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 13

name=Sally%20Strongar m&email=strongarm%40example.com
```

CASCADING STYLE SHEETS (CSS)

- **Utilisé pour la présentation**
  - Fonts
  - Couleurs
  - Images de fond
  - Interlignes
  - Disposition de la page etc…
- Avec CSS3, on peut ajouter des effets spéciaux et quelques animations à la page

http://www.csszengarden.com/

EXEMPLE

```html
<html>
    <head>
        <meta charset="utf-8">
        <title>Insert title here</title>
        <link href="NewFile.css" rel="stylesheet" type="text/css" />
    </head>
    <body>

        <h1> Titre de niveau 1 </h1>
        Bonjour
        <h1 class="italie"> Second Titre de niveau 1 </h1>
        Bonjour
        <h1 id="souligne"> Titre de niveau 1 souligne </h1>
        Bonjour
        <h1> Titre de niveau 1 final </h1>
        Au revoir
    </body>
</html>
```

NewFile.css

```css
h1 {
    color: blue;
}

h1.italie {
    font-style: Italic;
}

h1#souligne {
    text-decoration: underline;
}
```

Comment apparaissent les différents titre h1?

Polling

EXEMPLE

NewFile.css | NewFile.html | Insert title here

file:///Users/bgaudou/Dev/wsGAMA/HTML/NewFile.html

Titre de niveau 1

Bonjour

Second Titre de niveau 1

Bonjour

Titre de niveau 1 souligne

Bonjour

Titre de niveau 1 final

Au revoir

```
<h1> Titre de niveau 1 </h1>
Bonjour
<h1 class="italie"> Second Titre de niveau 1 </h1>
Bonjour
<h1 id="souligne"> Titre de niveau 1 souligne </h1>
Bonjour
<h1> Titre de niveau 1 final </h1>
Au revoir
```

NewFile.css

```
h1 {
    color: blue;
}

h1.italie {
    font-style: Italic;
}

h1#souligne {
    text-decoration: underline;
}
```

Polling

LES CLASSES

- Les classes : **nouveaux styles**, qu'il est possible d'affecter à **différentes balises**.
- Le sélecteur est limité au nom de la classe précédé de . :

```
.nom_classe {
   propriété: valeur1;
   propriété2: valeur2;
}
```

- Exemple : pour définir une classe ".important" qui met les éléments auxquels elle s'applique en rouge et gras :

```
.important {
   color: red;
   font-weight: bold;
}
```

2. LES CLASSES

- L'appel d'une classe peut être faite à partir de n'importe quelle balise HTML pour laquelle la définition est valide à l'aide de l'attribut **`class`**:

```
<balise class="nom_classe">
   ...
</balise>
```

- Exemple :

```
<h1 class="important"> Titre important </h1>
<em class="important">texte emphase important</em>
<span class="important">texte non-balisé important</span>
```

JAVASCRIPT

- JavaScript est un langage de script utilisé pour ajouter de **l'interactivité** et des **comportements** aux pages Web
  - Vérification des entrées de formulaire pour les entrées valides
  - Permuter des styles pour un élément ou un site entier
  - Création de widgets d'interface, tels que l'extension des menus
  - Modifier des éléments particuliers de la page (en utilisant la structure DOM de la page).

VALIDATION DE FORMULAIRES

```
<html>
<head>
   <title>Page Simple</title>
   <script language="JavaScript" type="text/javascript"
           src="validationFormulaire.js"></script>

</head>

<body>
 <form name="bonDeCommande" method="get"
       action="confirmation_correction.html">
   <input name="zone_pre" type="text" id="prenom">

   <input name="submit" type="submit" value="Envoi"
       onClick="return confirm('Etes-vous ?') && validation('prenom');">

 </form>
</body>
</html>
```

VALIDATION DE FORMULAIRES

- Element par ID

```
<html>
<head>
   <title>Page Simple</title>
   <script language="JavaScript" type="text/jav
           src="validationFormulaire.js"></scri

</head>

<body>
 <form name="bonDeCommande" method="get"
       action="confirmation_correction.html">
   <input name="zone_pre" type="text" id="prenom">

   <input name="submit" type="submit" value="Envoi"
       onClick="return confirm('Etes-vous ?') && validation('prenom');">

 </form>
</body>
</html>
```

```
function validation(id_zone_txt){
   zone_txt = document.getElementById(id_zone_txt)

   if(zone_txt.value == "Tintin"){
      alert("Erreur");
      return false;
   } else {
      return true;
   }
}
```

PROGRAMMATION COTÉ SERVEUR

- Certains sites Web simples sont des collections de documents HTML statiques et de fichiers image, mais la plupart des sites commerciaux ont des fonctionnalités plus avancées telles que la gestion des formulaires, les pages générées dynamiquement, les paniers, les systèmes de gestion de contenu, les bases de données, etc.
- Ces fonctions sont gérées par des applications Web exécutées sur le serveur.
  - **PHP**
  - **Python (Django, Flask)**
  - Ruby
  - **Node.js**
  - **Java** (Spring/SpringBoot, Google Web Toolkit,)
  - ASP.Net

PROGRAMMATION WEB DYNAMIQUE

Benoit Gaudou - benoit.gaudou@ut-capitole.fr

Source : Laurent Perrussel

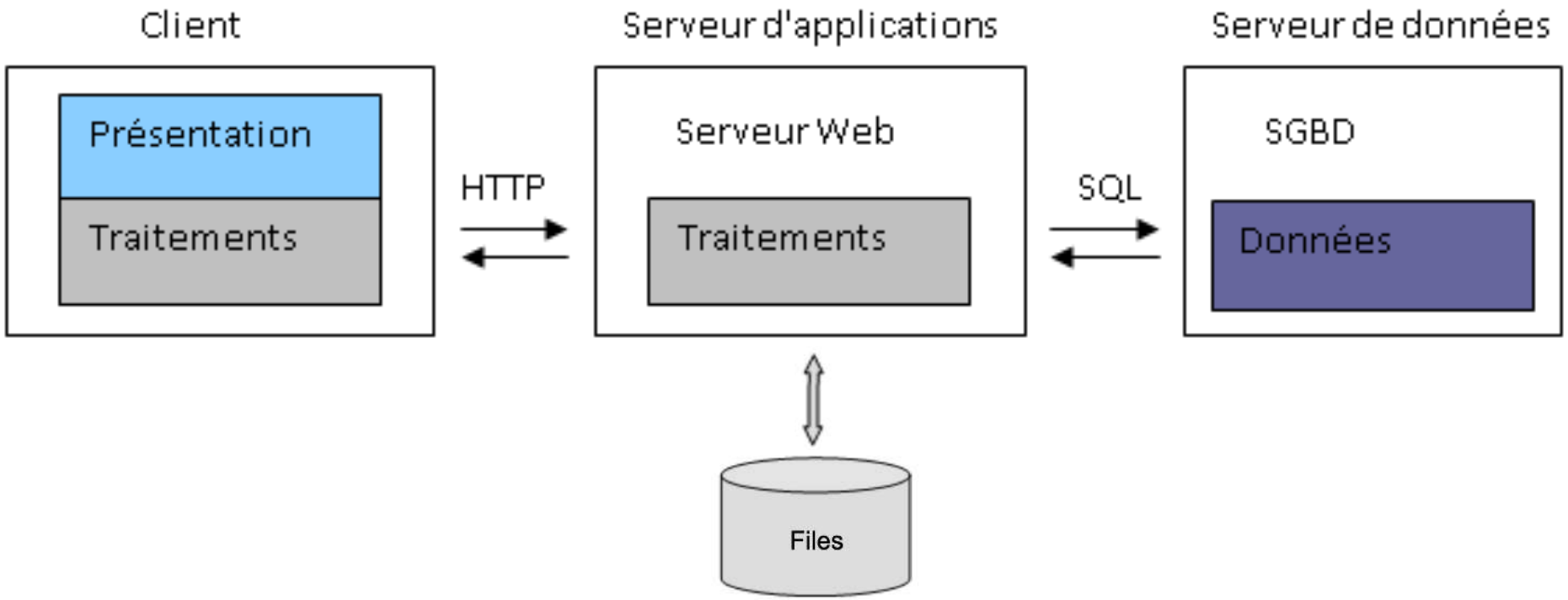
STATIC VS DYNAMIC WEBSITES: DEFINITIONS

- **Site web statique**
  Site web composé de pages HTML prédéfinies, créées une fois pour toutes à l'aide d'un éditeur HTML. Le contenu des pages est fixe.

- **Site web dynamique**
  Site web dont les pages HTML sont construites au fur et à mesure de la consultation par un internaute, à partir de bases de données et d'outils de mise en forme.
  Des sources de données externes peuvent également être utilisées pour construire les pages.

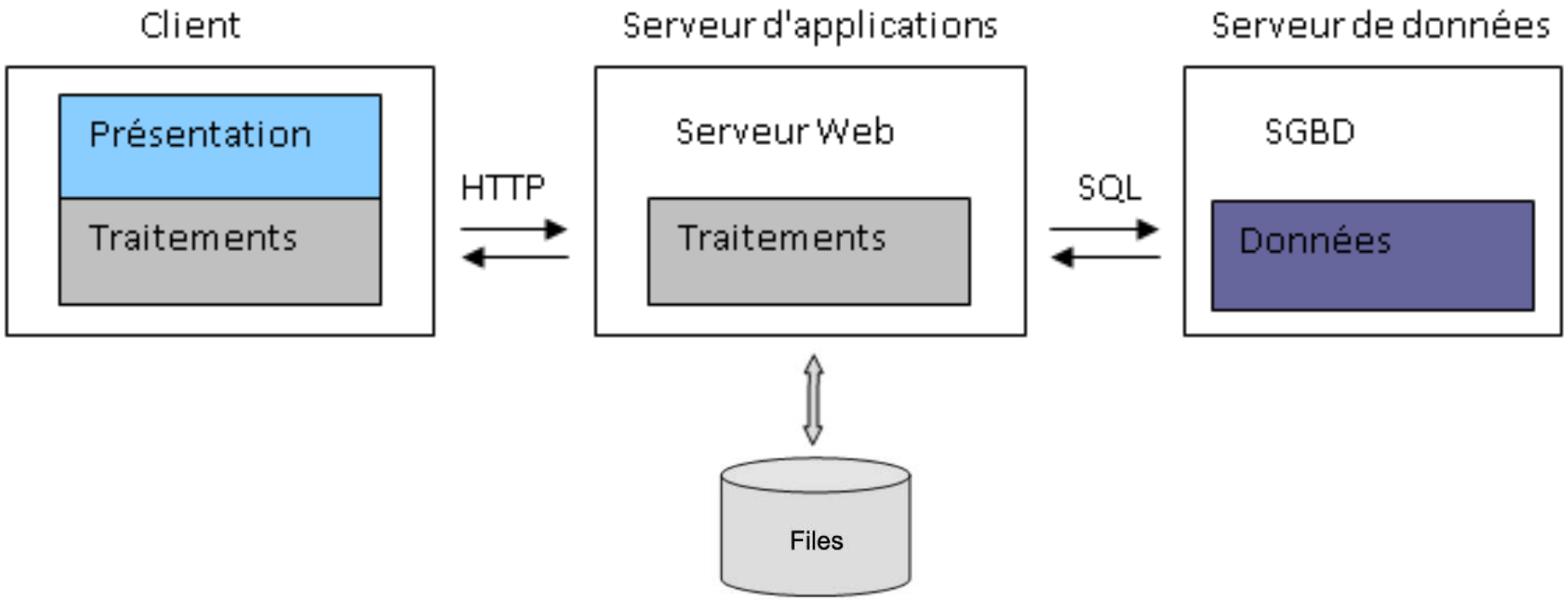
Source : http://www.journaldunet.com

3-TIER ARCHITECTURES



Note : les sites vont chercher également des données via des API (communication HTTP)

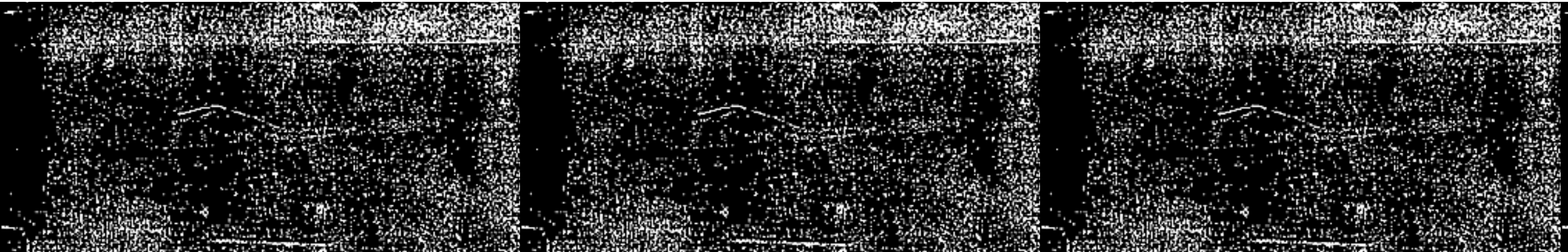
FRONTEND VS BACKEND



FrontEnd
HTML,CSS,Javascript

BackEnd
Java (Springboot)+SQL

APPROCHE À BASE DE FICHIERS (PHP)



APPROCHES POUR CONSTRUIRE DES SITES DYNAMIQUES

1. Utilisation de code intégré à une page HTML (PHP, JSP ...)
2. Utilisation de services web (Rest) (Flask, SpingBoot...)

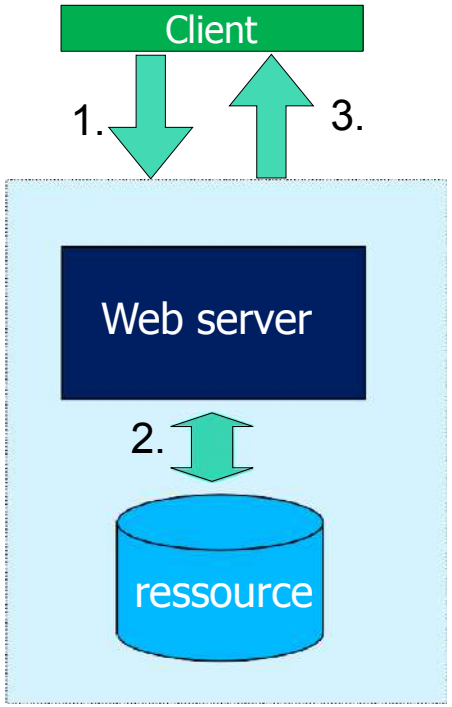
Envoi d'informations par un client

- via un formulaire
- via des scripts JavaScript

- Quelle que soit la méthode de génération, **la page HTML reçue par le client ne contient pas le code qui l'a générée.**

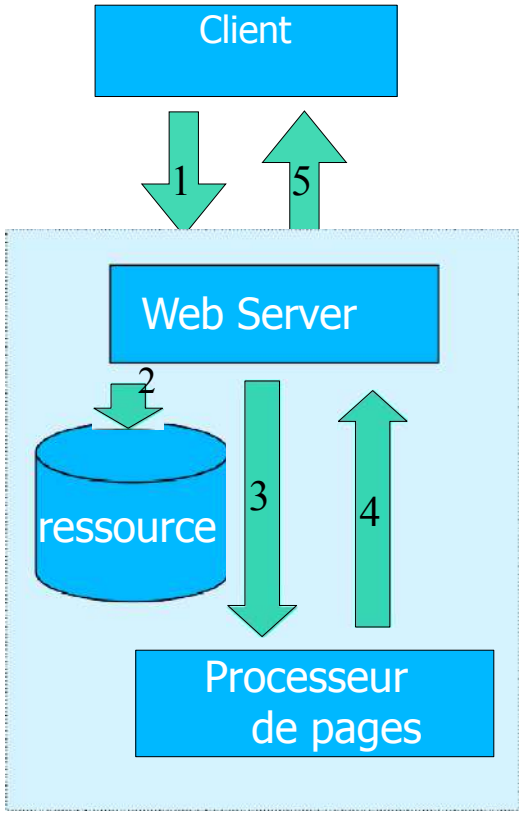
COMMENT FONCTIONNE UN SERVEUR WEB (STATIQUE)

1. Reçoit une requête HTTP
   - La requête mentionne la ressource demandées
2. Le serveur lit la ressource (le document) depuis le disque
3. La ressource est transmise au client



GENERATION OF A WEBSITE

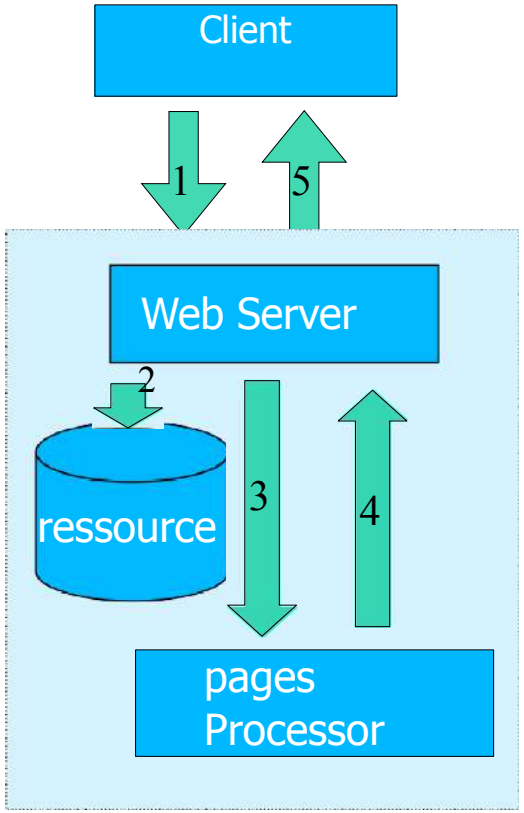
1. Réception de la requête HTTP
2. (Si nécessaire) Le serveur lit la ressource qui sera traité.
3. Transmission au processeur de pages
4. Instructions pour produire la nouvelle page sont traitées et la page résultante est envoyée au serveur web
5. La page complète est envoyée au client



GENERATION D'UNE PAGE HTML AVEC PHP

Fichier demandé par le client (disponible dans les ressources)

```
<!DOCTYPE html>
<html>
<body>

<h1> Bonjour </h1>
<?php
   $txt = "PHP";
   echo "I love $txt!";
?>

</body>
</html>
```



Fichier renvoyé par le serveur

```
<!DOCTYPE html>
<html>
<body>

<h1> Bonjour </h1>

I love PHP!

</body>

</html>
```

Affichage dans le navigateur

Bonjour

I love PHP!

ENVIRONNEMENT LOGICIEL

Client - presentation	Application server	Database server
• Browser	• Web server + dynamic pages processor	• SGBD

Apache Tomcat

LAMP/MAMP/WAMP

MariaDB